

Project NEXA — LangGraph + AWS Step Functions (Bedrock-enabled)

Generated: 2026-08-24 16:00 UTC **AWS account:** **Region:** ap-south-1 **Model:** `global.anthropic.claude-haiku-4-5-20251001-v1:0` (Bedrock cross-region inference profile) **Routing mode:** Bedrock-driven (`USE_BEDROCK=true`) — the LLM selects each next tool **Result:** SUCCEEDED — full end-to-end run including human approval resume

What was created

- DynamoDB table `nexa-langgraph-checkpoints` (graph state + task token storage)
- Step Functions state machine `nexa-langgraph-state-machine`
- Lambda `nexa-langgraph-intake` (implements the LangGraph node sequence up to the human-approval pause; calls Bedrock Converse to decide each next tool)
- Lambda `nexa-langgraph-execute` (resumes after approval, executes the transaction and audit tools)
- Lambda `nexa-langgraph-datapower-mock` (same 8-tool mock target)
- CloudWatch log group for the state machine (ALL logging level)

Two routing modes

The LangGraph Lambda supports two routing modes, controlled by the `USE_BEDROCK` environment variable:

1. **Deterministic mode** (`USE_BEDROCK=false`): the graph follows a hardcoded tool order (`extract_fields` → `retrieve_sop` → `run_validations` → `search_similar_cases` → `create_decision_task`). No LLM is invoked. An earlier successful run used this mode (`CASE-REPORT-001`).
2. **Bedrock-enabled mode** (`USE_BEDROCK=true`): at each graph node, the Lambda calls Bedrock Converse with the current state and asks the model to choose exactly one next tool from the available tool set. The model decides the sequence. This report describes a Bedrock-enabled run (`CASE-BEDROCK-002`).

Workflow

1. `StartExecution` with `case_id` and `case_text`.
2. Intake state invokes the intake Lambda with `arn:aws:states:::lambda:invoke.waitForTaskToken`. The Lambda runs the LangGraph node loop: at each step it asks Bedrock to select the next tool,

- calls the mock DataPower Lambda to execute that tool, appends the result to the graph state, and repeats. After `create_decision_task` is selected, the Lambda stores the Step Functions task token in DynamoDB and returns `{"status": "PENDING"}` — **blocking** on the task token. This is a structural, infrastructure-enforced human approval boundary that cannot be skipped by the model.
3. The task token is written to DynamoDB alongside the case state.
 4. An external caller retrieves the token and calls `SendTaskSuccess` with the decision (Approved).
 5. Execute state invokes the `execute` Lambda with the case id and decision, which calls `execute_transaction` and `write_audit_record`.
 6. Execution reaches `SUCCEEDED`.

Real execution result (Bedrock-enabled run)

Execution ARN: `arn:aws:states:ap-south-1:<AWS_ACCOUNT_ID>:execution:nexa-langgraph-state-machine:bedrock-1787583767`

Field	Value
Execution name	bedrock-1787583767
Case ID	CASE-BEDROCK-002
Case text	A broker emailed a change to the group contacts.
Start	2026-08-24T11:02:48
Stop	2026-08-24T11:08:58
Status	SUCCEEDED
Final output	<code>{"case_id":"CASE-BEDROCK-002","decision":"Approved","status":"CLOSED"}</code>

The ~6 minute elapsed time includes the period the execution waited at the `waitForTaskToken` state for the external approval callback.

Bedrock-selected tool sequence (intake phase)

The intake Lambda called Bedrock at each node. The model selected the following sequence:

#	Tool selected by Bedrock	Mock result
1	<code>extract_fields</code>	Redacted DOB; extracted address and tax ID
2	<code>retrieve_sop</code>	Required checks: address, tax ID, duplicate
3	<code>run_validations</code>	Address valid; tax ID valid; duplicate check skipped
4	<code>search_similar_cases</code>	

#	Tool selected by Bedrock	Mock result
5	create_decision_task	Two approved similar cases returned task_id: TASK-123, status: PENDING, review link

After create_decision_task, the Lambda stored the task token in DynamoDB and returned {"case_id":"CASE-BEDROCK-002","task_id":"TASK-123","status":"PENDING"}.

Intake Lambda metrics: Duration ~6894 ms, Billed ~7164 ms, Memory 512 MB.

Approval and resume (execute phase)

The task token was retrieved from DynamoDB and SendTaskSuccess was called with {"case_id":"CASE-BEDROCK-002","decision":"Approved"}. The state machine resumed and invoked the execute Lambda, which called:

#	Tool	Mock result
6	execute_transaction	{"status": "APPROVED", "message": "contacts updated"}
7	write_audit_record	{"status": "recorded"}

The execute Lambda returned {"case_id":"CASE-BEDROCK-002","decision":"Approved","status":"CLOSED"} and the state machine reached SUCCEEDED.

Execute Lambda metrics: Duration ~1021 ms, Billed ~1331 ms, Memory 512 MB.

CloudWatch log evidence

Intake Lambda (/aws/lambda/nexa-langgraph-intake):

```
[LANGGRAPH INTAKE] invoked case_id=CASE-BEDROCK-002
task_token_prefix=AQCIAAAAKgAAAAMA...
[LANGGRAPH INTAKE] _run_graph starting case_id=CASE-BEDROCK-002
case_text=A broker emailed a change to the group contacts.
[LANGGRAPH] _agent decided next tool: extract_fields
(use_bedrock=True)
[LANGGRAPH] _call_tool: extract_fields
[LANGGRAPH] _call_tool result (from mock): {"dob": "**REDACTED**",
"address": "123 Main St", "tax_id": "TX-9981"}
[LANGGRAPH] _agent decided next tool: retrieve_sop
(use_bedrock=True)
```

```

[LANGGRAPH] _call_tool: retrieve_sop
[LANGGRAPH] _call_tool result (from mock): {"required_checks":
["address", "tax_id", "duplicate"]}
[LANGGRAPH] _agent decided next tool: run_validations
(use_bedrock=True)
[LANGGRAPH] _call_tool: run_validations
[LANGGRAPH] _call_tool result (from mock): {"address": "valid",
"tax_id": "valid", "duplicate": "skipped"}
[LANGGRAPH] _agent decided next tool: search_similar_cases
(use_bedrock=True)
[LANGGRAPH] _call_tool: search_similar_cases
[LANGGRAPH] _call_tool result (from mock): {"similar_cases":
["CASE-0421", "CASE-0588"], "count": 2}
[LANGGRAPH] _agent decided next tool: create_decision_task
(use_bedrock=True)
[LANGGRAPH] _call_tool: create_decision_task
[LANGGRAPH] _call_tool result (from mock): {"task_id": "TASK-123",
"status": "PENDING", "review_link": "https://example.com/review/
TASK-123"}
[LANGGRAPH INTAKE] returning: {"case_id": "CASE-BEDROCK-002",
"task_id": "TASK-123", "status": "PENDING"}

```

Execute Lambda (/aws/lambda/nexa-langgraph-execute):

```

[LANGGRAPH EXECUTE] invoked case_id=CASE-BEDROCK-002
decision=Approved
[LANGGRAPH] _call_tool: execute_transaction
[LANGGRAPH] _call_tool result (from mock): {"status": "APPROVED",
"message": "contacts updated"}
[LANGGRAPH] _call_tool: write_audit_record
[LANGGRAPH] _call_tool result (from mock): {"status": "recorded"}
[LANGGRAPH EXECUTE] returning: {"case_id": "CASE-BEDROCK-002",
"decision": "Approved", "status": "CLOSED"}

```

Key implementation findings

1. **Lambda handler packaging** initially failed with `Runtime.ImportModuleError: Unable to import module 'index': No module named 'index' because the Terraform handler configuration (index.handler) did not match the actual module names in the deployment archives (langgraph_lambda.py). Fixed by pointing the handlers at langgraph_lambda.intake_handler / langgraph_lambda.execute_handler.`
2. **waitForTaskToken is intentionally blocking.** The Step Functions execution pauses at Intake until an external `SendTaskSuccess/SendTaskFailure` call is made — this cannot be bypassed by the model, in contrast to the Strands harness run where an unscripted model proceeded straight through the equivalent step. This is the key architectural difference: even with Bedrock-driven routing, the human-approval boundary is enforced by the workflow, not by the model's judgement.
3. **Bedrock client cache collision.** The first Bedrock-enabled attempt failed with `'function' object has no attribute 'converse'` because the module-level cache variable and the factory function were both named `_bedrock`. The

function definition overwrote the cached client, so `_bedrock()` returned the function object instead of a boto3 client. Fixed by renaming the cache variable; the subsequent run succeeded.

4. **Model naming.** The model selected `search_similar_cases` (matching the tool name as presented in the decision prompt) rather than `check_similar_cases` (the mock's internal name). The mock Lambda does not strictly validate tool names, so both resolve to the same handler. In a production system, tool-name matching should be exact.
5. Step Functions logging was set to ALL to capture full state transitions for debugging.

Comparison: deterministic vs Bedrock-enabled routing

Aspect	Deterministic (<code>USE_BEDROCK=false</code>)	Bedrock-enabled (<code>USE_BEDROCK=true</code>)
Tool selection	Hardcoded order in the graph	LLM chooses each next tool via Bedrock Converse
Model invoked	No	Yes (<code>global.anthropic.claude-haiku-4-5-20251001-v1:0</code>)
Tool sequence	Fixed: <code>extract</code> → <code>SOP</code> → <code>validate</code> → <code>similar</code> → decision	Same sequence, but model-selected
Human approval	Enforced by <code>waitForTaskToken</code>	Enforced by <code>waitForTaskToken</code>
Cost	Lambda + Step Functions only	Lambda + Step Functions + Bedrock inference
Latency	Lower (no model calls)	Higher (one Bedrock call per node)

Both modes produce the same tool sequence for this simple case, but the Bedrock-enabled mode demonstrates that the graph can delegate routing decisions to the model while still enforcing the human-approval boundary structurally.

AWS resources destroyed after test

The DynamoDB table, state machine, all three Lambdas, IAM roles, and CloudWatch log groups were destroyed immediately after the run completed. The state machine was deleted via the AWS CLI (Terraform destroy timed out waiting for the asynchronous deletion) and verified absent. No NEXA resources remain in the account.